E-commerce Backend System Using FastAPI

1. Project Overview

This E-commerce Backend System is a scalable, modular, and production-ready backend application built using FastAPI. It serves as the backbone of a complete e-commerce platform, handling core functionalities such as user authentication, product management, shopping cart, order processing, and role-based access control.

The system is designed with clean architecture principles and asynchronous programming to ensure high performance, security, and scalability,

User Roles

a. User (Customer)

- Can register, login.
- Can browse products listed by admin.
- Can add/remove items to/from cart.
- Can view and update cart items.
- Can checkout and place orders.
- Can view their order history.

b. Admin

- Can register, login.
- Can view All products.
- Can view their created products.
- Can **create**, **update**, and **delete** any product.

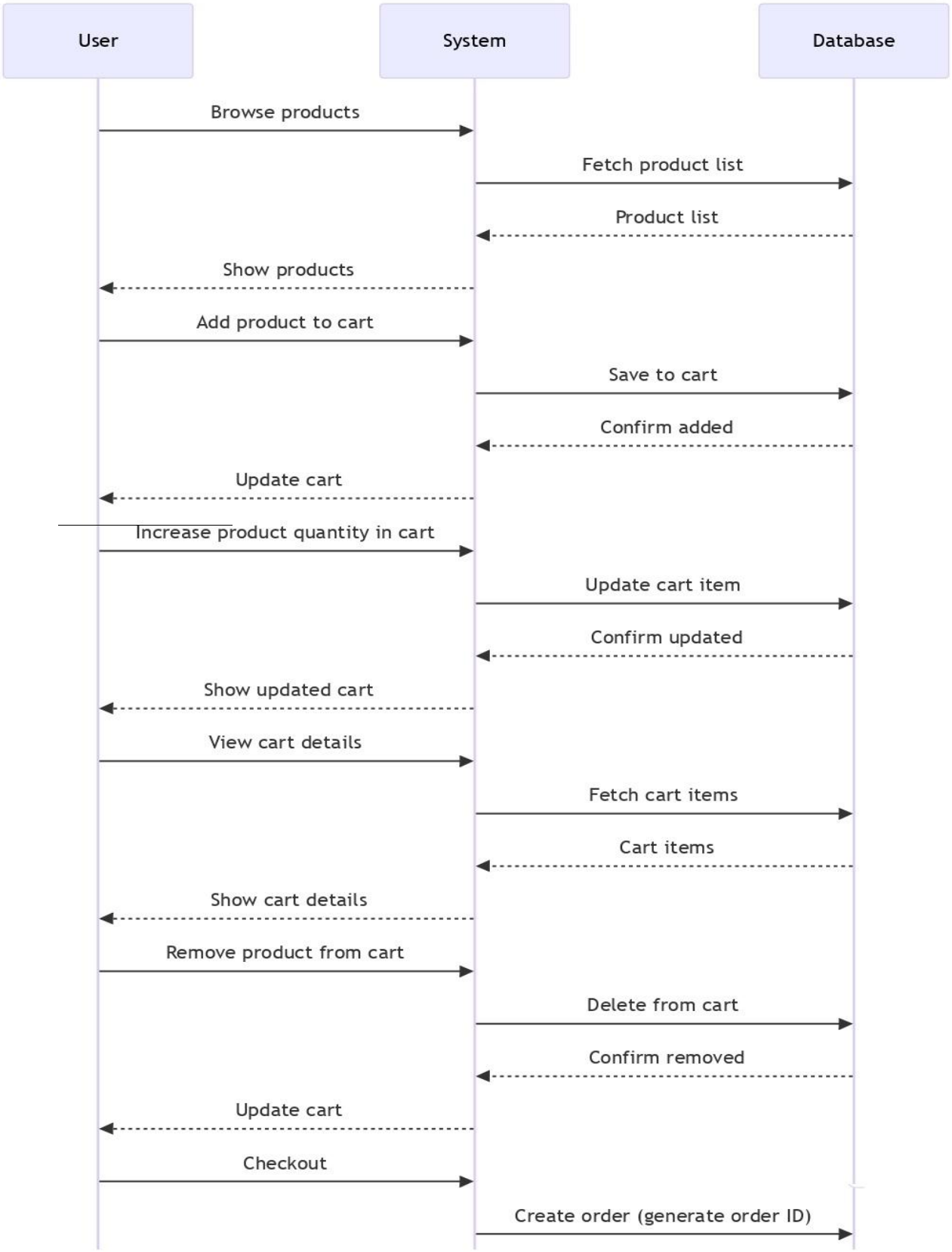
2. Tach Stack

- Language: Python 3.13.4
- Framework: FastAPI
- ORM: SQLAlchemy (Async)
- Database: PostgreSQL
- Authentication: OAuth2 with JWT (access & refresh tokens)
- Mail Service: SMTP

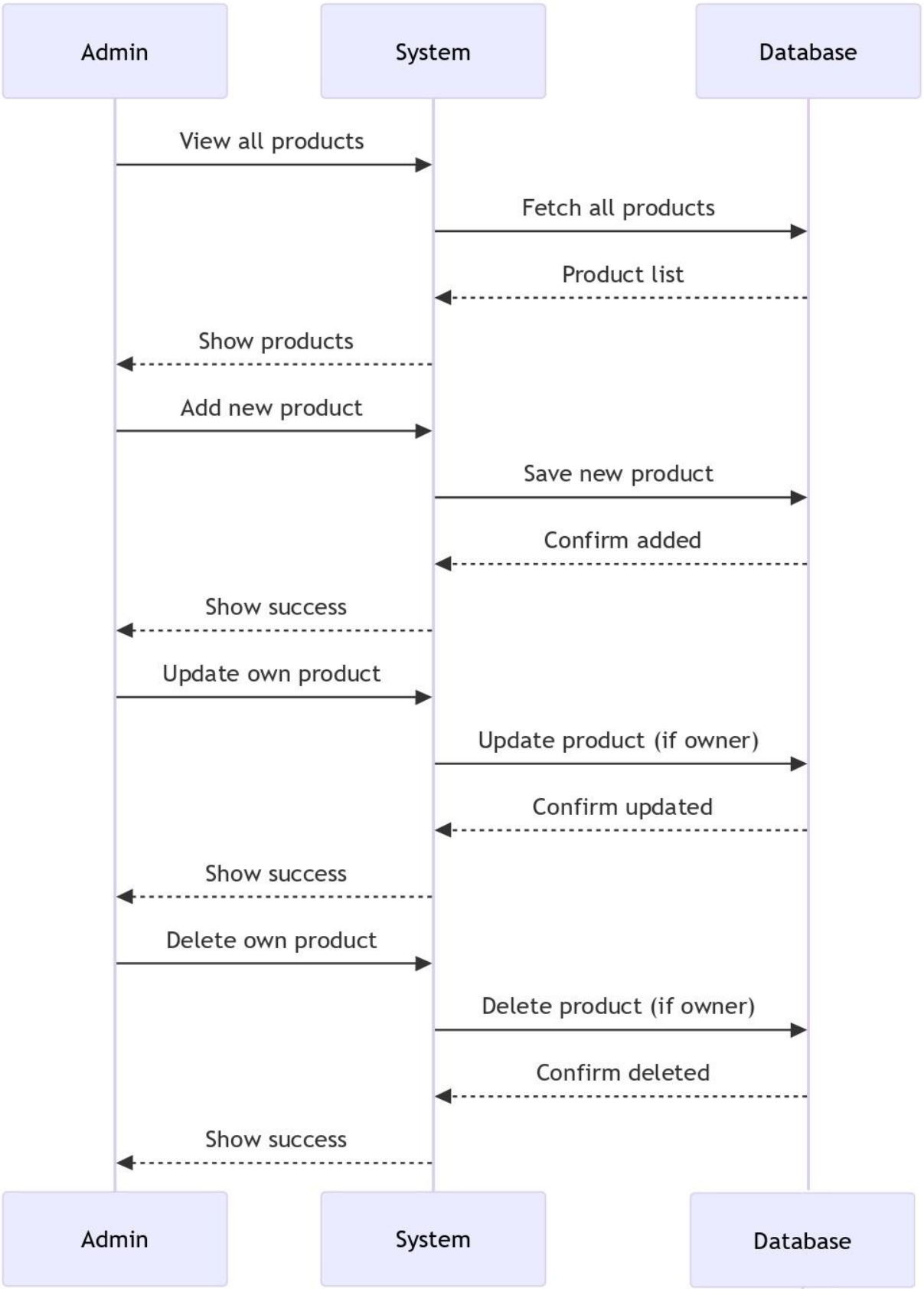
3. System Design

3.1 Sequence Diagram

User Flow

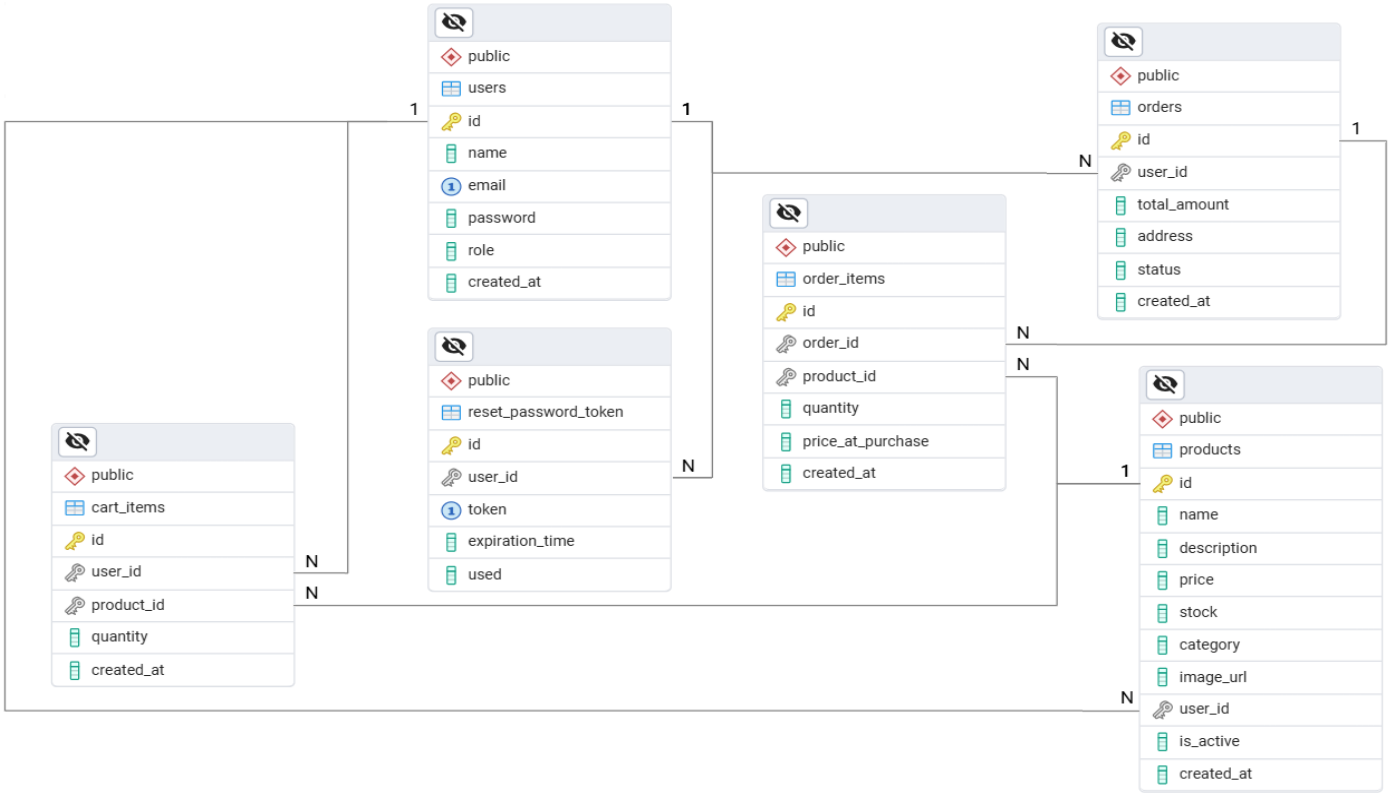


Admin Flow


Admin
System
Database
View all products
Fetch all products
Product list
Show products
Add new product
Save new product
Confirm added
Show success
Update own product
Update product (if owner)
Confirm updated
Show success
Delete own product
Delete product (if owner)
Confirm deleted
Show success
Admin
System
Database

3.2 Database Diagram

- **users**: Stores user details with fields like name, email, password, role, and created_at.
- **products**: Contains product-specific information with fields like name, description, price, stock, category, image_url, is_active, and a foreign key user_id (admin who created it).
- **cart_items**: Tracks items added to the cart with fields like user_id, product_id, quantity, and created_at.
- **orders**: Manages order records such as user_id, total_amount, address, status, and created_at.
- **order_items**: Stores item-level details within an order like order_id, product_id, quantity, price_at_purchase, and created_at.
- **reset_password_token**: Handles password reset flow with fields like user_id, token, expiration_time, used, and id.



4. Project Structure

app/
├── main.py
├── dependency.py
├── exceptions.py
├── global_exception_handler.py

```
├── configuration/
│   ├── cloudinary.py
│   ├── config.py
│   ├── database.py


│   ├── logger_config.py
├── auth/
│   ├── routes.py
│   ├── models.py
│   ├── schemas.py
│   ├── utils.py
├── products/
│   ├── models.py
│   ├── user/
│   │   ├── routes.py
│   │   ├── schemas.py
│   ├── admin/
│   │   ├── routes.py
│   │   ├── schemas.py
├── cart/
│   ├── routes.py
│   ├── models.py
│   ├── schemas.py
├── orders/
│   ├── routes.py
│   ├── models.py
│   ├── schemas.py
├── middlewares/
│   ├── oauth2.py
│   ├── route.py
│   ├── schema.py
├── utils/
│   ├── send_email.py
│   ├── password_reset_email.py
│   ├── reset_password_token.py
│   ├── image_uploader.py
│   ├── password_encoder.py
```

4. API Endpoints

HTTP Method	Endpoint	Description
POST	/auth/signup	Register a new user.
POST	/auth/signin	Login and generate access/refresh token.
POST	/auth/refresh-token	Get new access token using refresh token.
POST	/auth/forgot-password	Request password reset via email.
POST	/auth/reset-password	Reset password using token.
POST	/admin/products	Create a new product. (Admin only)
GET	/admin/products	View all products. (Admin only)
GET	/admin/products/{id}	Get product details by ID. (Admin only)
PUT	/admin/products/{id}	Update a product. (Admin only)
DELETE	/admin/products/{id}	Delete a product. (Admin only)
GET	/products	View all products. (User)
GET	/products/search?keyword=	Search products by keyword. (User)
GET	/products/{id}	View product details. (User)
POST	/cart	Add product to cart. (User)
GET	/cart	View cart items. (User)
PUT	/cart/{product_id}	Update quantity of product in cart. (User)
DELETE	/cart/{product_id}	Remove product from cart. (User)
POST	/checkout	Dummy checkout, clears cart and creates an order. (User)
GET	/orders	View order history. (User)
GET	/orders/{order_id}	Get details of a specific order. (User)

6. Features and Functionalities

- **User Authentication:** Secure authentication using FastAPI and JWT.
- **Role-Based Access:** Role-based actions for Admin and Users.
- **Product Management:** Admin can manage product inventory.
- **Cart System:** Users can manage cart items including add, update, remove.

- **Order Processing:** Orders created on checkout with product details.
- **Email Support**: Reset password functionality using email tokens.

7. Implementation Details

7.1 User Management

- **Registration:** Users can register with their basic details. Role is assigned as 'user' by default.
- **Login**: Users log in using email and password, receive JWT tokens.

7.2 Product Management

- **Create Product:** Admin can create new products.
- **Update Product:** Admin can update details of their own products.
- **Delete Product:** Admin can remove their own products.
- **View Products**: Admin and users can view product list and details.

7.3 Cart Management

- **Add to Cart**: Users can add products to their cart.
- **Update Cart**: Users can increase or decrease quantity of cart items.
- **Remove from Cart:** Individual products can be removed.
- **View Cart:** Users can view their cart details.

7.4 Order Management

- **Checkout:** On checkout, products from the cart are added to an order.
- **Order Items:** Stores product-wise order info with price snapshot.
- **Order History:** Users can view past orders and their contents.

7.5 Password Reset

- **Forgot Password**: Users can request password reset via email.
- **Reset Password:** Token-based reset system with expiry support.

7.6 Authentication and Authorization

- **FastAPI Security**: Secured endpoints using FastAPI dependencies.
- **JWT Tokens:** Tokens used for authentication and session control.
- **Role-based Access:** Admins can manage products; users can browse, order, and manage cart.
- **BCrypt:** Passwords are encrypted using bcrypt hashing.

8. Conclusion

This E-commerce Backend System built using FastAPI provides secure and scalable product browsing, cart handling, and order processing functionalities. It leverages role-based access for user and admin roles, along with secure authentication mechanisms. The system is ready for integration with a frontend or mobile app.